

**BEACON LABS**

CYBERSECURITY EDUCATION LABS

# **State Inference Attack on Android Lab**

**Module: Mobile Security**



**BOISE STATE UNIVERSITY**

**COLLEGE OF ENGINEERING**

*Department of Computer Science*

## Table of Contents

|                                                                                                     |    |
|-----------------------------------------------------------------------------------------------------|----|
| Overview .....                                                                                      | 3  |
| Setup .....                                                                                         | 4  |
| Opening the Lab Project .....                                                                       | 4  |
| Creating the Emulation Device .....                                                                 | 4  |
| Task Set 1: Game Theory .....                                                                       | 6  |
| Task Set 2: Socially Engineer Application to Trick User and MPM .....                               | 8  |
| Task 2.1: Tricking User .....                                                                       | 8  |
| Task 2.2: Stealing Credentials .....                                                                | 10 |
| Task Set 3: Accessing State Information via Usage Stats Manager<br>Through Background Service ..... | 11 |
| Task 3.1: Permission for Usage Stats Manager .....                                                  | 11 |
| Task 3.2: Implementing Usage Stats Manager .....                                                    | 12 |
| Task Set 4: Hijack Activity via View Intents Through Injection .....                                | 13 |
| Submission .....                                                                                    | 14 |
| Summary .....                                                                                       | 14 |
| Trouble Shooting .....                                                                              | 15 |
| MPM isn't recommending logins to save on certain device/applications .....                          | 15 |
| Refactoring the Project Name .....                                                                  | 15 |
| References .....                                                                                    | 15 |

## Overview

When people think about phishing attacks, many commonly think of emails or fake links that target company email domains. However, a very prevalent form of phishing attacks target mobile devices via instant apps and applications. These applications monitor the root device and infer when to inject an application.

This exercise will build upon the research paper, *Detecting and Preventing State Inference Attacks* by Possemato et al.<sup>[1]</sup>, covering vulnerabilities and strategies surrounding mobile devices. In this lab, you will implement and socially engineer an application to monitor state information of a device and steal user information.

## Objectives

Understand conventional attack strategies on mobile devices and some of the vulnerabilities within Android API's, how to utilize vulnerable API's to gather state information about a device and its processes, and how to implement a social engineering application that hijacks an activity and steals user information.

## Reading

This lab is based on a paper by Andrea Possemato, Dario Nisi, and Yanick Fratantonio published in the 2021 Network and Distributed System Security (NDSS) Symposium <sup>[1]</sup>. Reading the original paper is not necessary but can help if you would like to learn more about this attack and defense.

A. Possemato, D. Nisi and Y. Fratantonio, "Preventing and Detecting State Inference Attacks on Android", 2021.<sup>[1]</sup>

## Setup

A provided zipped folder contains all necessary files in order to run and complete this lab. It is recommended that you have 8 GB RAM or more, x86\_64 CPU architecture; 2nd generation Intel Core or newer, or AMD CPU with support for a Windows Hypervisor / ARM-based chips, or 2nd generation Intel Core or newer with support for Hypervisor.Framework, and 9 GB of available disk space minimum or more.

## Opening the Lab Project

Download the 'Lab.zip' file from the Project Folder.

Direct link to Project Folder: [https://drive.google.com/drive/folders/1UxNYI3COA3WzHXo\\_deNatGNIVjSYyzvA?usp=sharing](https://drive.google.com/drive/folders/1UxNYI3COA3WzHXo_deNatGNIVjSYyzvA?usp=sharing)

Once downloaded, unzip the 'Lab.zip' file and then in Android Studio, open the 'Lab/Android Studio Project' project.

## Creating the Emulation Device

On the right hand side of your screen, click the 'Device Manager' tab, and select 'Create device'. Scroll down and select the 'Nexus 5X' option. When choosing your system image, select the second tab that corresponds to your CPU architecture (x86 / ARM). Scroll down and select API 27 for Android Oreo. Make sure your application binary interface corresponds to your matching architecture. Lastly, make sure beneath the target column that your system image **does not** utilize Google API's or services.

Now that you have created your device, run the device and go back into your project folder and drag and drop each package into your actively running emulator to install the necessary applications for this lab. Lastly, to conclude the setup for our lab, run your emulated device and create an account with the provided mobile password manager, *Keeper*. It is recommended that you use your school email and a password that you will remember. Go into the application settings and provide access to Keeper AutoFill through following the prompts. Additionally, make sure to increase

the time of its Auto Logout from 10 minutes to 30 minutes to increase the convenience of our testing.

## Task Set 1: Game Theory

In this game theory model, there are two players: Player 1, who is the attacker, and Player 2, who represents the API's that are being enumerated/invoked. Each API has been designed such that depending on the circumstances or variables that the API is called within, the response the API gives us changes. These variables are unknown to the attacker, and thus a sufficient strategy must be used to successfully discover any potentially vulnerable API's. Player 2, or the API, of course is informed and aware of the necessary variables and types needed to correctly invoke itself. Ideally, as the attacker, your job is to successfully reduce the amount of API's as much as possible to find a list of truly vulnerable API's in order for you to retrieve state information from a target device. Utilize figure 1 on page 7 to answer the following questions.

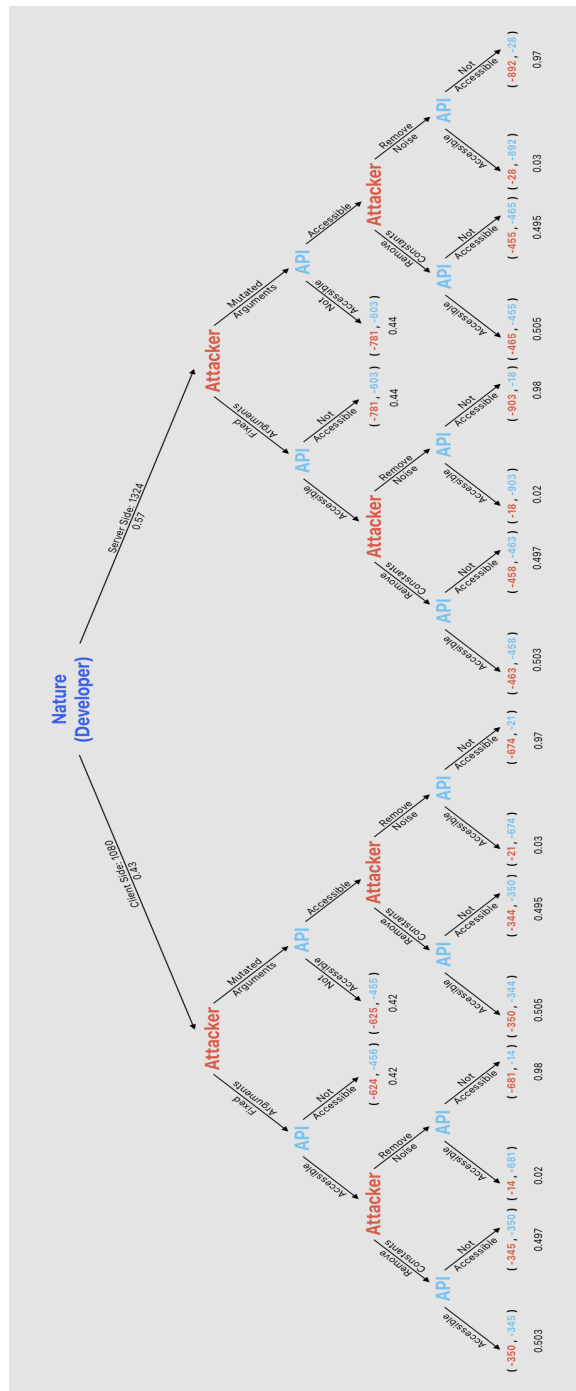
### Action

1.1.1 - Based on the given game tree, which outcome can you infer has the most utility, and can you make an assumption as to why that might be based on the project description?

1.1.2 - Based on the given game tree, what is one example of an adverse selection when enumerating API's?

1.1.3 - Utilizing Bayes' Theorem, what is the probability that an API was invoked via removing noise through mutated arguments on the client side?

1.1.4 - Utilizing Bayes' Theorem, what is the probability that an API was invoked via removing constants through static arguments on the client side?



## Task Set 2: Socially Engineer Application to Trick User and MPM

### Task 2.1: Tricking User

The idea behind social engineering is to convince and trick a suspected user into believing that an application is legitimate. If a service appears to be legitimate, it is much more likely a user will feel comfortable providing sensitive information to an attacker or an application. The technique behind this is through modeling an application as close as possible. Click the green play button at the top of the IDE and explore the device environment for yourself a little bit.

#### Action

2.1.1 - What common theme do you notice between some of the third-party applications installed on the device?

2.1.2 - What would be the best part or page of an application to mimic or trick a user and steal user information?

If you notice, all of the third-party applications that are installed on our emulated device are shopping applications. Navigate to the log-in page of any one of these applications and take note of the layout. It is recommended that you take a screenshot of this page for future reference. An example of this is noted in figure 2. Our first objective is to replicate this page within our application, and we will do so by learning about Android's `activity_main.xml` file that allows us to control and change the UI of our application. It is very similar in practice to HTML or JavaScript.

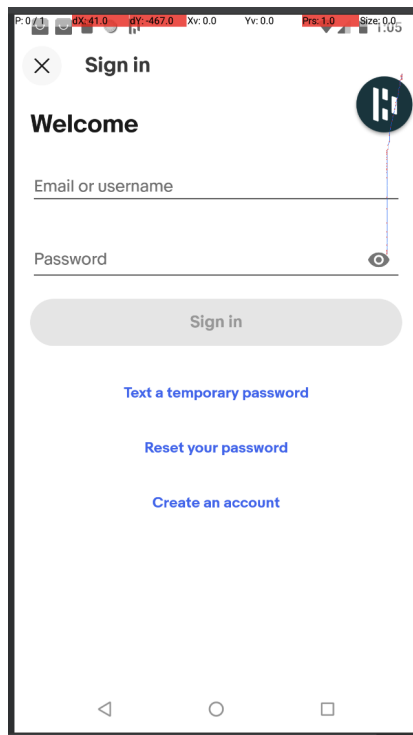


Figure 2: Example of login page

In your project window on the left-hand side, navigate to your 'res' package, and open the folder 'layout'. Beneath this, you will find your activity\_main.xml file. Once you open this, you will notice in the top right corner of your viewport that you have the option to change between your code and the design viewport. Here, you will implement any respective buttons, input fields, or images to match the log-in page from the target application as much as possible. You will notice that you have a skeleton outline for a few primitive operations to get you started on utilizing these features, but any additional features or solutions can be found on the [Android Studio UI documentation](#).

### Action

2.1.3 - Create your replicated attack application and submit a screenshot of both your replicated version and the original application.

## Task 2.2: Stealing Credentials

```
E/ContentValues: onTick: 49
I/PHISHING_ATTACK: Email: johndoe@gmail.com Password: test12345
D/EGL_emulation: eglMakeCurrent: 0x7e402f9e60: ver 3 0 (tinfo 0x7e354df160)
D/AutofillManager: notifyViewVisibilityChanged(): id=1073741824 isVisible=true
D/AutofillManager: notifyViewVisibilityChanged(): id=1073741825 isVisible=true
E/ContentValues: onTick: 48
```

*Figure 3: Example of credentials stolen from application*

Now that you have successfully created a page that could potentially trick a user, now it is time to trick the MPM into suggesting credentials and stealing them. Navigate back to your 'MainActivity.java' file. You will need to implement an [event listener](#) that will pull the credentials at an optimal time that were entered on your page, such as figure 3. This would typically be implemented after the 'log in' button is pressed. The trick to input fields and tricking the MPM is through a design tool known as [autofill-Hints](#). Through your user-input fields, create a variable that will collect and store the inputs provided by the user and print them out to the console to validate that your method works.

### Action

2.2.1 - Create an input field within your 'activity\_main.xml' file that utilizes autofillHints to suggest what type of input to provide. Submit a screenshot of your element.

2.2.2 - Create an on click listener function within your 'MainActivity.java' class that steals credentials from the input field. Submit a screenshot of your code.

2.2.3 - Utilize the debug log to print out the stolen credentials to validate that your code performed as expected. Submit a screenshot of the output.

## Task Set 3: Accessing State Information Via Usage Stats Manager Through Background Service

### Task 3.1: Permission for Usage Stats Manager

First things first - we need to provide our application access to the Usage Stats Manager. This section will teach you about [View Intents](#) and set you up for how to perform an injection in the next section. While it might seem silly to implement an attack that requires certain signature-level permissions, many real-world apps (both benign and malicious) currently use them, thus it is considered an appropriate threat model. First, it is smart for us to create a method that validates whether or not a permission has been previously provided as it will one improve the speed of our testing and two better trick the user into believing its validity once approved.

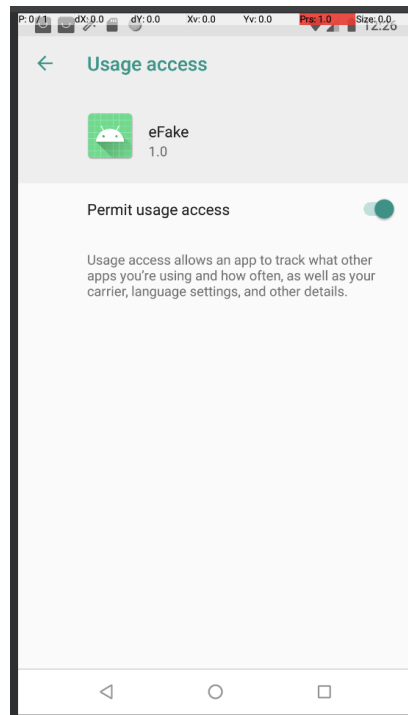


Figure 4: Example of USM permission window

Navigate to 'MyService.java' and create a method at the start of your attack function within your runnable to check and see if the permission is granted as noted in figure 4.

### Action

3.1.1 - Create a short section within 'MyService.java' that checks whether or not Usage Stats Manager has been granted permission and submit a screenshot of your code.

## Task 3.2: Implementing Usage Stats Manager

```
E/ContentValues: onTick: 51
I/System.out: com.android.providers.calendar
I/System.out: com.alibaba.aliexpresshd
I/System.out: com.dashlane
I/System.out: com.android.defcontainer
I/System.out: com.android.launcher3
I/System.out: com.etsy.android
I/System.out: com.android.inputmethod.latin
I/System.out: org.chromium.webview_shell
I/System.out: com.zzkko
I/System.out: com.android.settings
I/System.out: com.ebay.mobile
E/ContentValues: onTick: 50
```

*Figure 5: Example of USM package list*

Once you have enabled [Usage Stats Manager](#), you will have access to the contained public methods that will allow you to access state information on the device. As you can see in figure 5, this is an example output of active app packages in between tick intervals. Read through the short documentation notes and utilize one of the different public methods for Usage Stats Manager to return a list of information about all of the active packages. Note that each public method will return different types of information that can be used in a similar manner to determine state information.

**Action**

3.2.1 - Now that you have permission to Usage Stats Manager, create a function that returns a list of activities and submit a screenshot of your code. Your output should be similar to Figure 5 on page 5 above.

## Task Set 4: Hijack Activity via View Intents Through Injection

Here, you will combine techniques learned in the previous modules to inject your activity. Through the list of services you identified via the Usage Stats Manager, choose a target application to monitor that is different than the application whose login page you selected to replicate. Do this by isolating the target application from the list returned from Usage Stats Manager and create conditional logic that identifies when that application is opened (in other words, moved to the foreground), and, using View intents, inject your application as if it were prompting the user to log in. Finally, once the user clicks the log in button on your replica page, utilize a separate View intent to return the user back to the original application they tried to open as if they never left it in the first place.

Furthermore, to convince the MPM that your application is legitimate, refactor your project to have an identical package name as the one you replicated. Thanks to increased security measures, Android prevents two applications from containing the same package name. By doing this, you will either replace or need to delete the previous application in order to successfully refactor your project. This is why it is even more important to replicate the original application as closely as possible. Do this by changing your application's package name to match your target application's package name within 'build.gradle (:app)' and 'AndroidManifest.xml'. These can be found under the 'Gradle Scripts' and 'manifests' folders respectively beneath your project window. Once you've changed those, select 'File' -> 'Invalidate caches...' -> 'Restart', and your project should correctly rebuild. Note that you will likely have to reprovide permission to Usage Stats Manager, given that your application is considered a new app since

it has a different package name.

**Action**

4.1.1 - Create a section that isolates your target application through monitoring the USM list and injects your malicious application via a view intent. Submit a screenshot of your code.

4.1.2 - Once you have injected your application, create a section that returns the user back to the target application after you've intercepted their credentials in a suspected 'log in' attempt. Submit a screenshot of your code.

## Submission

Submit a lab report utilizing the provided template to your instructor, which should include screenshots, thoughts, challenges, and lessons learned from the lab. Furthermore, zip and attach your project files with the relevant java files for submission.

## Summary

This lab explored conventional attack strategies and mobile vulnerabilities to monitor and collect state information about a device. It also utilized social engineering and tools to trick a mobile password manager and user into providing sensitive information to be collected by an attacker.

## Trouble Shooting

### MPM isn't recommending logins to save on certain devices/application

- Log into MPM and store the password combination yourself

### Refactoring the Project Name

- Make sure you change the respective lines within build.gradle and AndroidManifest.xml to match what you refactored your project to. Then, select File->Invalidate Caches->Restart, and your project should rebuild with the correct dependencies. If this does not work, you can select Build->Clean Project and Build->Rebuild Project and then retry invalidating the caches. This will fully correct your problem.

## References

- [1] Andrea Possemato, Dario Nisi, and Yanick Fratantonio. Preventing and detecting state inference attacks on android. In *NDSS*, 2021.